QoS-Aware AI Inference Offloading for Vehicular Edge Computing: A Physics-Based Dataset and Simulation Framework

Konstantinos Gyftodimos, Eirini Liotou, Kyriakos Chiotis, George Dimitrakopoulos
Dept. of Informatics and Telematics, Harokopio University of Athens (HUA), Athens, Greece
{kgyftodimos, eliotou, kchiotis, gdimitra}@*hua.gr* *Presenter: K. Gyftodimos — onsite*

***Abstract*—We present a physics-based open-source simulation framework for QoS-aware AI inference offloading in vehicular edge computing, comprising a 50,000-sample synthetic dataset across five road scenarios, a multi-output machine-learning predictor, and a real-time closed-loop demonstration.**

I. Overview

Autonomous vehicles generate continuous high-rate video streams that must be processed by YOLOv8 [1] object detection under a strict 100 ms end-to-end latency constraint. The simulation framework models a four-tier execution hierarchy: (A) a dedicated lightweight on-board neural processing unit (NPU), (B) the vehicle's primary high-throughput GPU, (C) a nearby Multi-Access Edge Computing (MEC) [2] server, and (D) a remote cloud data centre. Each tier runs a different YOLOv8 variant and offers a distinct latency–accuracy trade-off (Table I). Selecting the optimal tier in real time requires jointly predicting network conditions and enforcing Quality of Service (QoS) constraints simultaneously.

Three properties of the vehicular setting make that decision hard. The radio link is non-stationary: mobility, NLOS blockage, and cell-edge handovers change the achievable uplink rate on sub-second timescales, so a decision taken on stale state can miss the deadline. The MEC server is contended, so its queueing delay depends on neighbouring vehicles the ego-vehicle cannot observe. And the tiers differ in accuracy as well as in latency, so protecting the latency budget can silently degrade perception—a safety concern, not merely a performance one.

Most vehicular offloading studies rely on proprietary radio traces, or abstract the channel so far that MEC latency variability is not faithfully reproduced. This artifact provides an open, reproducible baseline—from 3GPP-calibrated channel simulation to closed-loop optimisation—that is self-contained, runs offline, and ports to any compute-intensive edge inference workload.

Delivery format: onsite demonstration; the system runs as a single terminal command on a standard laptop, with no external hardware or network.

II. Innovation & Contribution

1) **Physics-based synthetic dataset (50 k samples).** Generated via a 3GPP TR 38.901-calibrated log-distance path-loss model [3] with log-normal shadowing and non-line-of-sight (NLOS) transitions, combined with Shannon–Hartley throughput [4]. Five road scenarios span 300 m to 10 km MEC inter-site distances (urban dense through rural), with scenario-specific speed, bandwidth, and frequency profiles. *How the data is acquired:* no proprietary or measured trace is used—every row is synthesised. A scenario is drawn from a fixed prior, then speed, MEC distance, co-served UEs (Poisson, rush-hour scaled), server load (Beta), payload, and WAN congestion are sampled from scenario-conditioned distributions and pushed through a deterministic chain: path loss $\rightarrow$ RSSI (log-normal shadowing, 25–45 dB NLOS penalty) $\rightarrow$ SINR $\rightarrow$ Shannon rate. RTT labels sum uplink, propagation, load-dependent queueing, and a 20 ms handover penalty; reachability is labelled by an SINR threshold. Labels are generated, not annotated, and the table regenerates from seed 42 in under a minute (MIT license; schema in Sec. V).
2) **Multi-output machine-learning (ML) predictor.** A `RandomForestRegressor` (200 trees, 80/20 train-test split) predicts MEC and cloud round-trip time (RTT) and server capacity utilisation; a `GradientBoostingClassifier` predicts MEC availability (binary). All scikit-learn [5] pipelines are serialised to disk after training and loaded on subsequent runs, reducing cold-start time to $\approx$1 s. Non-stationarity is handled at decision time rather than by the regressor alone: an exponential moving average ($\alpha = 0.3$) damps jitter, while spikes the model cannot anticipate—abrupt NLOS blockage, a handover—are absorbed by the closed loop, which re-evaluates every frame at 5–10 Hz, so a mispredicted frame costs one frame interval rather than a persistent mode error. Reachability is a separate classifier, so a hard outage is a discrete event and not an extrapolated latency, and a shipped 90th-percentile model (`predictor_q90.pkl`) can replace the mean predictor for conservative gating. Tracking genuine drift would still need online re-fitting; the training entry point is exposed for that.
3) **Edge-first QoS optimizer.** The decision engine enforces three hard QoS budgets: total latency $\leq$100 ms, accuracy loss $\leq$15%, and server capacity $<$95%. Among feasible

TABLE I
YOLOv8 EXECUTION MODES: SPECIFICATION AND LIVE OPTIMIZER SNAPSHOT (SUBURBAN, FRAME 100/100, MEC DIST. 611 M, SIGNAL −55.5 DBM, 4 MODE SWITCHES). TOT. (MS) = INFERENCE + NETWORK RTT.

M.	Hardware / Model	Tot. (ms)	mAP	AccLoss	Status
A	Car NPU / `yolov8n`	8.0	37.3	20.8%	× loss > 15%
B	Car GPU / `yolov8n`	5.0	37.3	20.8%	× loss > 15%
C	MEC / `yolov8m`	56.9	50.2	4.9%	◀ **SELECTED**
D	Cloud / `yolov8l`	71.6	52.9	0.0%	feasible, not preferred

Budget: latency ≤100 ms, acc. loss ≤15%, cap. <95%.
Modes A/B exceed the accuracy budget; emergency fallback only.

modes it applies an edge-first preference rule: Mode C (MEC) is chosen over Mode D (Cloud) when the cloud model's incremental mean Average Precision (mAP) gain is below 5 pts, conserving Wide Area Network (WAN) bandwidth. If no mode satisfies all constraints—e.g. during a connectivity outage—the optimizer falls back to Mode B (on-board GPU) with an explicit reason flag (`fallback_no_feasible_mode`). Mode B is deliberately allowed to break the accuracy budget: the budgets are ranked lexicographically, with latency and reachability held as hard safety constraints and accuracy loss the one that yields when nothing is feasible—through a prolonged outage, degraded detection beats none. The violation is never silent: the flag is raised on every affected frame, the signal a production stack forwards to the planner to reduce speed or extend headway until a compliant tier returns.

4) **Live four-tier failover demo.** One command exercises all four tiers in a 100-frame run: an injected outage (frames 40–54) drives a full C → B → C failover, making every QoS constraint path observable in real time.

III. Relevance to CloudNet

The artifact targets CloudNet researchers working on vehicular edge computing [6], task offloading [7], and 5G edge AI [8]. It contributes a **reproducible baseline**—a 3GPP-calibrated offloading dataset over five deployment archetypes, from dense urban (300 m MEC spacing) to rural (10 km)—and **evidence that offloading is necessary**: on-board YOLOv8n loses 20.8% accuracy against cloud YOLOv8l, breaching the 15% budget whenever a remote tier is reachable. It is also an **end-to-end blueprint** coupling ML network-state prediction to a policy-based QoS optimizer, portable to other workloads (LiDAR, V2X perception) and extendable with new tiers or policies.

IV. Demonstration Plan & Requirements

Setup. Run `pip install -r requirements.txt`, then `python demo.py`. The first run auto-generates the 50 k dataset and trains all models (≈50 s); later runs load the serialised models and start in ≈2 s.

What the audience observes. The terminal view (Table I) refreshes at 5 Hz for 100 frames, showing all four tiers at once with live per-mode latency, accuracy loss, capacity, and decision reason. In normal suburban conditions Mode C is selected over Mode D by the edge-first rule, with Modes A and B visible and marked infeasible. At frame 40 the injected outage forces MEC unavailable and cloud RTT to 200 ms; the optimizer drops to Mode B with reason `fallback_no_feasible_mode`, then recovers.

AV / Power: standard laptop (CPU-only), terminal with ANSI colour; no special power, networking, or AV equipment needed.

V. Artifact Details

All components are in a public GitHub repository (https://github.com/icsa-hua/CLOUDNET2026-QoS-Offloading, release `v1.0.0`; Ubuntu 22.04 / macOS 14 / Python 3.11). The **dataset** is a 50 k-row CSV regenerated by `python -m src.dataset.generate --seed 42`. Its 13 observable features cover the scenario label, vehicle speed, time of day, MEC distance, RSSI, channel bandwidth, MEC load, co-served UEs, handover flag, payload size, WAN congestion, cloud base RTT, and cloud load; the 5 targets are MEC reachability (binary), MEC and cloud RTT (ms), and MEC and cloud capacity (%). The **framework** splits into `src/dataset/` (channel sim), `src/predictor/` (ML pipelines), and `src/offloading/` (QoS optimizer); dataset and models are bundled and auto-recreated if absent. The **demo** (`python demo.py`) is configurable via `--scenario`, `--hz`, and `--outage-at`.

VI. Ethics, Privacy & Safety

The dataset is entirely synthetic—generated from a calibrated radio-physics model with no real-world measurements, user data, or personally identifiable information. No human subjects or consent procedures are involved; the framework poses no safety risk.

Acknowledgment

This work has been funded by the Shift2SDV project (Grant Agreement No. 101194245), supported by the Chips Joint Undertaking and its members, including top-up funding by the national authorities of Austria, Denmark, Germany, Greece, Finland, Italy, Netherlands, Poland, Portugal, Spain, and Turkey.

References

[1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. https://github.com/ultralytics/ultralytics

[2] ETSI, "GS MEC 003: MEC framework and reference architecture, v3.1.1," 2022.

[3] 3GPP, "TR 38.901: Channel model for frequencies from 0.5 to 100 GHz, v16.1.0," 2019.

[4] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, 1948.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[7] X. Chen *et al.*, "Optimized computation offloading via deep reinforcement learning," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4005–4018, 2019.

[8] Y. Liu *et al.*, "Toward edge intelligence: MEC for 5G and IoT," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6722–6747, 2020.